

HABITAT Protocol — local TypeScript implementation

This package is a standalone implementation of the Habitat spatial-memory protocol. It is not the visual web app. It provides a typed World State, command parser, validation, mutation, Journal events, and JSON persistence that can run locally with Node.js.

Requirements

Use Node.js 20 or newer. The package uses TypeScript, `tsx`, and Vitest.

Install and run

Bash

```
npm install
npm run demo
```

The interactive CLI creates `habitat-state.json` in the current directory. Set `HABITAT_STATE` to choose another path.

Bash

```
HABITAT_STATE=./data/habitat.json npm run demo
```

You can also execute one command without entering interactive mode:

Bash

```
npm run demo -- STATE
npm run demo -- OBSERVE
npm run demo -- INSPECT plant
npm run demo -- MOVE wine_leather_chair TO L2/E6
npm run demo -- FOCUS L2
npm run demo -- TOGGLE lamp OFF
npm run demo -- JOURNAL LAST 5
```

Supported commands

Command	Effect
---------	--------

<code>OBSERVE</code>	Records an observation without changing object positions.
<code>INSPECT <object></code>	Returns the object schema and records an observation.
<code>MOVE <object> TO <level>/<cell></code>	Validates and commits a move, or rejects it without mutation.
<code>FOCUS <L1 L2 ALL></code>	Changes the active visual focus. <code>SHOW</code> is accepted as an alias.
<code>TOGGLE <object> <ON OFF></code>	Changes a toggle-capable object, such as the L1 lamp.
<code>JOURNAL LAST <n></code>	Returns the latest Journal entries.
<code>STATE</code>	Returns the current World State as JSON.

Coordinates use levels `L1` and `L2`, columns `A-J`, and rows `1-10`. A move is rejected when the object does not exist, is fixed, has an invalid coordinate, attempts a cross-level relocation, or targets an occupied cell. Rejected actions leave object positions unchanged but still create a rejected Journal event.

Development checks

Bash

```
npm run check
npm test
```

The test suite covers command parsing, accepted moves, collision rejection, fixed-object rejection, focus, inspection, toggling, observation, Journal output, and JSON round trips.

Architecture

`src/index.ts` is the protocol engine. `WorldState` is the source of truth. `execute(state, input, actor)` is the single state-transition entry point: it parses a command, validates it, creates a normalized Journal event, and returns the next state plus a human-readable response. `src/cli.ts` is a thin persistence-aware command-line adapter.